

claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-ac11c11436fac2f74.jsonl`  
- SHA-256: `396d9ddd63a209ebdf1d4ebd248b28a6388c2f03b836b5d3c3ead872d633aaca`  
- Source modified: `2026-06-10T16:28:23+00:00`  
- Imported at: `2026-07-05T16:48:21+00:00`  
- project: `wf_27df53c8-8e0`  
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`
```

Transcript

1. user (2026-06-10T16:25:50.955Z)

You are an adversarial verifier. A code reviewer audited the repo at C:/Users/avidu/Projects/gmail-attachments-to-gdrive and reported this finding:

TITLE: Backfill advances lastSynced past windows that had attachment-level errors, so failed messages are never retried
SEVERITY CLAIMED: high
FILE: src/sync.ts (line 213-218)
DESCRIPTION: After each window, lastSynced is advanced whenever the daily cap was not hit ? result.hasError is ignored. A message whose attachment copy threw (Drive 500, rate limit, 'service invoked too many times') is correctly left out of processedIds (drive.ts:174-180 promises 'the next run retries it'), but the 30-day window has already moved past it and backfill only walks forward. The next window's after: clause only reaches 1 day back, so unless the failure happened in the window's final day the message is permanently skipped. This bites hardest exactly when errors are most likely: if the self-imposed cap undercounts after a hard kill (see related finding) and the account's real ~250 files/day Drive-creation quota is exhausted, every createFile throws, hasError is set, the window still advances, and a whole window of attachments is lost.

```
EVIDENCE QUOTED: fullSynced = search.isToday;  
    if (added < MAX_FILES_ADD_PER_DAY) {  
        lastSynced = fullSynced  
        ? today.toJSON()  
        : new Date(Date.parse(lastSynced) + THIRTY_DAYS).toJSON();  
    }
```

SUGGESTED FIX: Do not advance lastSynced when result.hasError (or result.errors > 0) for the window; let the next run re-scan it ? processedSet and the getFilesByName probe make the re-scan cheap and duplicate-free. If a poison message could stall progress forever, add a bounded per-window retry count before advancing.

Your job is to try to REFUTE it. Your lens: reachability and impact: check whether the scenario can actually occur in realistic single-user use on Windows, and whether the assigned severity is justified.

Read the actual code in the repo (and tests/config where relevant). If after reading you cannot confirm the problem is real, set isReal=false. If it is real but over/under-stated, set adjustedSeverity accordingly. Be strict: plausible-sounding but unconfirmed = refuted.

SENSITIVE-DATA RULES (mandatory, highest priority):

- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/, testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file existence and git-tracking status (git ls-files, git log --all --<path>).
- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers, balances, person names, client secrets, transaction amounts) in your output.
- Never include sensitive values in your findings. Refer to them abstractly ("a real account

number", "a client_secret value").

- Do not use WebSearch/WebFetch and do not send any repo content to any external service.
Structured output only.

2. user (2026-06-10T16:26:00.006Z)

```
1      import {
2          ENABLE_INCREMENTAL_SYNC,
3          MAX_FILES_ADD_PER_DAY,
4          MAX_RUNTIME_MS,
5          MAX_SYNC_TRIGGERS,
6          ONE_DAY,
7          ONE_MINUTE,
8          SIX_HOURS,
9          THIRTY_DAYS,
10     } from "./config";
11     import { updateDrive, type UpdateResult } from "./drive";
12     import {
13         getCurrentHistoryId,
14         listAddedSince,
15         threadsForMessageIds,
16     } from "./gmailHistory";
17     import { logDebug, logError } from "./logger";
18     import { maybeSendDailySummary } from "./notify";
19     import { addProcessedId } from "./processed";
20     import { buildSearchQuery } from "./query";
21     import { hasTime, isSameCalendarDay } from "./quota";
22     import { dayKey, recordRun } from "./stats";
23     import * as state from "./state";
24
25     const SYNC_HANDLER = "syncUserDrives";
26
27     /** Schedules a one-shot run of syncUserDrives `afterMs` from now. */
28     export function newSyncTrigger(afterMs: number): void {
29         ScriptApp.newTrigger(SYNC_HANDLER).timeBased().after(afterMs).create();
30     }
31
32     /** All currently-installed triggers for the sync handler. */
33     function getSyncTriggers(): GoogleAppsScript.Script.Trigger[] {
34         return ScriptApp.getProjectTriggers().filter(
35             (t) => t.getHandlerFunction() === SYNC_HANDLER,
36         );
37     }
38
39     /** Removes every project trigger (used when the user re-runs setup). */
40     export function deleteAllTriggers(): void {
41         for (const trigger of ScriptApp.getProjectTriggers()) {
42             ScriptApp.deleteTrigger(trigger);
43         }
44     }
45
46     function deleteTriggers(triggers: GoogleAppsScript.Script.Trigger[]): void {
47         for (const trigger of triggers) {
48             ScriptApp.deleteTrigger(trigger);
49         }
50     }
51
52     /** Defense-in-depth: never let sync triggers exceed the cap (limit is 20). */
53     function pruneSyncTriggers(max: number): void {
54         const triggers = getSyncTriggers();
55         for (let i = max; i < triggers.length; i++) {
56             ScriptApp.deleteTrigger(triggers[i]);
57         }
58     }
59
```

```

60  /**
61   * The background worker. A per-user lock ensures only one run executes at a time
62   * for THIS user, so concurrent triggers cannot stack file copies or trigger
63   * generations. It is deliberately a user lock, not a script lock: the web app
64   * runs as the accessing user (executeAs USER_ACCESSING), so a script-wide lock
65   * would serialise every user's sync into one global queue.
66   */
67  export function syncUserDrives(): void {
68      const lock = LockService.getUserLock();
69      if (!lock.tryLock(0)) {
70          logDebug("Another sync run is in progress; skipping.");
71          return;
72      }
73      try {
74          runSyncOnce();
75      } finally {
76          // Bound the trigger count regardless of which path we took, then release.
77          pruneSyncTriggers(MAX_SYNC_TRIGGERS);
78          lock.releaseLock();
79      }
80  }
81
82  /**
83   * A single sync pass:
84   * 1. reschedules a fan-out of backup triggers (self-healing if one is lost),
85   * 2. syncs forward in 30-day windows until it runs out of time, hits the
86   *    daily file cap, or fully catches up,
87   * 3. persists progress and schedules the next run.
88   */
89  function runSyncOnce(): void {
90      const today = new Date();
91
92      // On the first run of a new calendar day, summarise the previous day.
93      maybeSendDailySummary(today);
94
95      // Recover state if the debug flag was cleared (e.g. properties were wiped).
96      const debug = state.getDebug();
97      if (debug == null || debug === "0") {
98          state.setDebug("1");
99          state.setLastSynced(new Date(today.getTime() - 365 * ONE_DAY).toJSON());
100      }
101
102      // Create the backup triggers BEFORE deleting the current ones, so a failure
103      // here can never leave us with no scheduled run.
104      const previousTriggers = getSyncTriggers();
105      try {
106          newSyncTrigger(60 * ONE_MINUTE);
107          newSyncTrigger(SIX_HOURS);
108          newSyncTrigger(2 * SIX_HOURS);
109          newSyncTrigger(4 * SIX_HOURS);
110          newSyncTrigger(8 * SIX_HOURS);
111          newSyncTrigger(28 * SIX_HOURS);
112      } catch (e) {
113          // Could not install new triggers; leave the old ones and try again later.
114          logError("Could not install triggers: " + e);
115          return;
116      }
117      deleteTriggers(previousTriggers);
118
119      const filesAddedToday = state.getFilesAddedToday();
120      let added = filesAddedToday.added;
121      const lastCountDate = new Date(Date.parse(filesAddedToday.date));
122
123      if (isSameCalendarDay(lastCountDate, today) && added >= MAX_FILES_ADD_PER_DAY) {
124          logDebug("Used up all quota for today. Can't add more files.");

```

```

125     newSyncTrigger(60 * ONE_MINUTE);
126     return;
127 }
128 if (!isSameCalendarDay(lastCountDate, today)) {
129     added = 0;
130 }
131
132 const structure = state.getFolderStructure();
133 let lastSynced = state.getLastSynced();
134
135 // Load the dedup set once; thread it through the run and persist at the end.
136 let processedIds = state.getProcessedIds();
137 const processedSet = new Set(processedIds);
138
139 // This run's contribution to the day's stats.
140 const addedAtStart = added;
141 let runErrors = 0;
142 let runLastError = "";
143
144 // Folds one updateDrive result into this run's accumulators.
145 const applyResult = (result: UpdateResult): void => {
146     added += result.addedFiles;
147     runErrors += result.errors;
148     if (result.lastError) {
149         runLastError = result.lastError;
150     }
151     for (const id of result.processedIds) {
152         if (!processedSet.has(id)) {
153             processedSet.add(id);
154             processedIds = addProcessedId(processedIds, id);
155         }
156     }
157 };
158
159 try {
160     // Incremental mode is opt-in (ENABLE_INCREMENTAL_SYNC) so it rolls out
161     // gradually over the proven date-window backfill. A stored historyId only
162     // exists once a backfill has caught up while the flag was on.
163     const historyId = ENABLE_INCREMENTAL_SYNC ? state.getHistoryId() : null;
164     if (historyId) {
165         // Incremental: process only messages added since the stored historyId.
166         const changes = listAddedSince(historyId);
167         if (changes == null) {
168             // History expired: clear the cursor and re-scan a bounded recent window
169             // (the next catch-up re-captures a fresh historyId). Overlap is safe ?
170             // dedup by message id and the Drive filename guard prevent re-copying.
171             state.clearHistoryId();
172             lastSynced = new Date(today.getTime() - THIRTY_DAYS).toJSON();
173             logDebug("History expired; reverting to date-window sync.");
174         } else {
175             if (changes.messageIds.length > 0 && added < MAX_FILES_ADD_PER_DAY) {
176                 const threads = threadsForMessageIds(changes.messageIds);
177                 if (threads.length > 0) {
178                     applyResult(
179                         updateDrive(
180                             threads,
181                             structure,
182                             MAX_FILES_ADD_PER_DAY - added,
183                             processedSet,
184                         ),
185                     );
186                 }
187             }
188             // Advance the cursor so the next run only sees newer changes.
189             state.setHistoryId(changes.historyId);

```

```

190     }
191   } else {
192     // Backfill: walk forward in 30-day windows until caught up to today.
193     // Capture the mailbox historyId BEFORE processing so mail arriving
194     // mid-run is still picked up by the first incremental scan.
195     const candidateHistoryId = ENABLE_INCREMENTAL_SYNC
196       ? getCurrentHistoryId()
197       : null;
198     let fullSynced = false;
199     while (
200       hasTime(new Date(), today, MAX_RUNTIME_MS) &&
201       added < MAX_FILES_ADD_PER_DAY &&
202       !fullSynced
203     ) {
204       const maxToAdd = MAX_FILES_ADD_PER_DAY - added;
205       const search = buildSearchQuery(lastSynced, new Date());
206       logDebug("Query: " + search.query + " files asking: " + maxToAdd);
207
208       const threads = GmailApp.search(search.query);
209       if (threads.length > 0) {
210         applyResult(updateDrive(threads, structure, maxToAdd, processedSet));
211       }
212
213       fullSynced = search.isToday;
214       if (added < MAX_FILES_ADD_PER_DAY) {
215         lastSynced = fullSynced
216           ? today.toJSON()
217           : new Date(Date.parse(lastSynced) + THIRTY_DAYS).toJSON();
218       }
219     }
220     // Caught up: switch future runs to the cheaper incremental path.
221     if (candidateHistoryId && fullSynced && added < MAX_FILES_ADD_PER_DAY) {
222       state.setHistoryId(candidateHistoryId);
223       logDebug("Backfill complete; stored historyId " + candidateHistoryId);
224     }
225   }
226 } catch (e) {
227   logError("Some error in execution: " + e);
228   state.setLastError(String(e));
229   runErrors++;
230   runLastError = String(e);
231 }
232
233 // Fold this run into the day's accumulator.
234 const dayStats = recordRun(state.getRunStats(dayKey(today)), {
235   filesAdded: added - addedAtStart,
236   errors: runErrors,
237   lastError: runLastError,
238 });
239 state.setRunStats(dayStats);
240
241 state.setProcessedIds(processedIds);
242 state.setLastSynced(lastSynced);
243 state.setFilesAddedToday({ added, date: today.toJSON() });
244 newSyncTrigger(20 * ONE_MINUTE);
245 }
246

```

3. user (2026-06-10T16:26:00.838Z)

```

1   import { GMAIL_ATTACHMENT_FOLDER } from "./config";
2   import { isAllowedContentType } from "./contentTypes";
3   import { getAttachmentName, getFolderNameForStructure } from "./naming";
4   import { logDebug, logError } from "./logger";
5

```

```

6     type Folder = GoogleAppsScript.Drive.Folder;
7
8     /**
9     * Anything that can hold folders: both `DriveApp` (the root) and a `Folder`
10    * expose `getFolders()` / `createFolder()`, so we accept either structurally.
11    */
12    interface FolderContainer {
13        getFolders(): GoogleAppsScript.Drive.FolderIterator;
14        createFolder(name: string): Folder;
15    }
16
17    export interface UpdateResult {
18        /** Number of files actually written to Drive in this call. */
19        addedFiles: number;
20        hasError: boolean;
21        /** Count of attachment-level errors in this call. */
22        errors: number;
23        /** Most recent error message, if any. */
24        lastError: string;
25        /** Gmail message ids fully handled in this call (for cross-run dedup). */
26        processedIds: string[];
27    }
28
29    /** Returns the named child folder, creating it if it does not exist yet. */
30    export function getOrCreateFolderInFolder(
31        parent: FolderContainer,
32        name: string,
33    ): Folder {
34        const folders = parent.getFolders();
35        while (folders.hasNext()) {
36            const folder = folders.next();
37            if (folder.getName() === name) {
38                logDebug("Found existing folder " + name);
39                return folder;
40            }
41        }
42        logDebug("Creating new folder " + name);
43        return parent.createFolder(name);
44    }
45
46    /**
47    * Resolves the destination sub-folder for an attachment, creating it under
48    * `head` and caching it in `existingFolders`. Returns null when the chosen
49    * structure yields no folder (caller falls back to the head folder).
50    */
51    function resolveFolder(
52        head: Folder,
53        existingFolders: Map<string, Folder>,
54        structure: string,
55        contentType: string,
56        date: Date,
57    ): Folder | null {
58        const name = getFolderNameForStructure(structure, contentType, date);
59        if (name === "") {
60            return null;
61        }
62        const cached = existingFolders.get(name);
63        if (cached) {
64            return cached;
65        }
66        const created = head.createFolder(name);
67        existingFolders.set(name, created);
68        return created;
69    }
70

```

```

71  /**
72   * Copies every allowed, not-yet-present attachment from the given threads into
73   * the GmailAttachments folder, organised per the user's chosen structure.
74   * Stops once `maxToAdd` files have been written.
75   *
76   * `alreadyProcessed` is the set of Gmail message ids handled in previous runs;
77   * such messages are skipped wholesale, avoiding a Drive `getFilesByName` probe
78   * per attachment when an overlapping date window is re-scanned. Ids fully
79   * handled here are returned in `processedIds` for the caller to persist.
80   */
81  export function updateDrive(
82    threads: GoogleAppsScript.Gmail.GmailThread[],
83    structure: string,
84    maxToAdd: number,
85    alreadyProcessed: ReadonlySet<string> = new Set(),
86  ): UpdateResult {
87    const head = getOrCreateFolderInFolder(DriveApp, GMAIL_ATTACHMENT_FOLDER);
88
89    // Pre-load existing sub-folders so we reuse rather than duplicate them.
90    const existingFolders = new Map<string, Folder>();
91    const currentFolders = head.getFolders();
92    while (currentFolders.hasNext()) {
93      const folder = currentFolders.next();
94      existingFolders.set(folder.getName(), folder);
95    }
96    logDebug("Already present folders: " + existingFolders.size);
97
98    const addedFiles = new Map<string, Folder>();
99    const processedIds: string[] = [];
100    let addMore = maxToAdd > 0;
101    let hasError = false;
102    let errors = 0;
103    let lastError = "";
104
105    for (const thread of threads) {
106      if (!addMore) break;
107      for (const message of thread.getMessages()) {
108        if (!addMore) break;
109        const messageId = message.getId();
110        // Skip messages we fully handled on a previous run.
111        if (alreadyProcessed.has(messageId)) {
112          logDebug("Skipping already-processed message " + messageId);
113          continue;
114        }
115        let messageFailed = false;
116        const from = message.getFrom();
117        // GmailMessage#getDate is typed as the stripped Base.Date interface but is
118        // a real JS Date at runtime.
119        const date = message.getDate() as unknown as Date;
120        const attachments = message.getAttachments();
121        let lastHandled = -1;
122        for (let a = 0; a < attachments.length && addMore; a++) {
123          const attachment = attachments[a];
124          lastHandled = a;
125          const contentType = attachment.getContentType();
126          const fileName = getAttachmentName(attachment.getName(), from, date);
127
128          if (!isAllowedContentType(contentType) || addedFiles.has(fileName)) {
129            logDebug("Rejected file of type: " + contentType);
130            continue;
131          }
132
133          let folder = resolveFolder(
134            head,
135            existingFolders,

```

```

136         structure,
137         contentType,
138         date,
139     );
140     if (folder == null) {
141         // Unknown structure: drop it in the top-level folder for recovery.
142         logError("Could not resolve directory; using top-level folder.");
143         folder = head;
144     }
145
146     try {
147         // Skip if a file with this name already exists in the folder.
148         if (folder.GetFilesByName(fileName).hasNext()) {
149             continue;
150         }
151         const blob = attachment.copyBlob();
152         blob.setName(fileName);
153         const file = folder.createFile(blob);
154         file.setDescription(
155             "Created from a Gmail Attachment from " +
156             from +
157             " sent on date " +
158             date,
159         );
160         addedFiles.set(fileName, folder);
161         Utilities.sleep(100);
162         if (addedFiles.size === maxToAdd) {
163             addMore = false;
164         }
165     } catch (e) {
166         logError("Attachment copy failed (message " + messageId + "): " + e);
167         hasError = true;
168         messageFailed = true;
169         errors++;
170         lastError = String(e);
171         Utilities.sleep(1000);
172     }
173 }
174 // Mark the message done only if we handled all its attachments cleanly.
175 // If we stopped early on the daily cap (lastHandled short of the end) or
176 // hit a transient failure, leave it unmarked so the next run retries it.
177 const completed = !messageFailed && lastHandled === attachments.length - 1;
178 if (completed) {
179     processedIds.push(messageId);
180 }
181 }
182 }
183
184 return { addedFiles: addedFiles.size, hasError, errors, lastError, processedIds };
185 }
186

```

4. assistant (2026-06-10T16:26:12.124Z)

Now let me check the query builder to verify the window-overlap claim, and the config.

5. user (2026-06-10T16:26:13.073Z)

```

1     import { ONE_DAY, THIRTY_DAYS } from "../config";
2
3     export interface SearchQuery {
4         /** The Gmail search expression. */
5         query: string;
6         /** ISO timestamp of the upper bound of this window (for debugging). */
7         before: string;

```



```

8      /** True once the window has reached "now" ? i.e. we are fully caught up. */
9      isToday: boolean;
10     }
11
12     /**
13      * Builds the Gmail search for the next 30-day window after `lastSynced`.
14      *
15      * `now` is injected (rather than read from the clock) so the windowing logic is
16      * deterministic under test. NOTE: this intentionally preserves the legacy
17      * behaviour of mixing a UTC upper bound (`Date.parse`) with local-time day
18      * parts (`getFullYear`/`getMonth`/`getDate`) when formatting the Gmail
19      * `before:`/`after:` strings, so the day boundaries follow the script's
20      * configured timezone.
21      */
22     export function buildSearchQuery(lastSynced: string, now: Date): SearchQuery {
23         let query = "to:me in:all has:attachment -in:trash -in:spam ";
24         let nextDate = THIRTY_DAYS + Date.parse(lastSynced);
25         const today = now.getTime();
26         let isToday = false;
27         if (today < nextDate) {
28             isToday = true;
29             nextDate = today;
30         }
31         nextDate += ONE_DAY;
32
33         const afterDate = new Date(Date.parse(lastSynced) - ONE_DAY);
34         const beforeDate = new Date(nextDate);
35         const beforeQ = `${beforeDate.getFullYear()}/${beforeDate.getMonth() +
1}/${beforeDate.getDate()}`;
36         const afterQ = `${afterDate.getFullYear()}/${afterDate.getMonth() +
1}/${afterDate.getDate()}`;
37
38         if (beforeQ === afterQ) {
39             // Window collapses to a single day; just search "after yesterday".
40             const yesterday = new Date(Date.parse(lastSynced) - ONE_DAY);
41             const yesterdayQ = `${yesterday.getFullYear()}/${yesterday.getMonth() +
1}/${yesterday.getDate()}`;
42             query = query + " " + "after:" + yesterdayQ;
43         } else {
44             query = query + "before:" + beforeQ + " " + "after:" + afterQ;
45         }
46
47         return { query, before: beforeDate.toJSON(), isToday };
48     }
49

```

6. user (2026-06-10T16:26:13.337Z)

```

1      /**
2       * The MIT License (MIT)
3       * Copyright (c) 2013 Avi Dullu
4       */
5
6      /** Top-level Drive folder that holds every synced attachment. */
7      export const GMAIL_ATTACHMENT_FOLDER = "GmailAttachments";
8
9      /** Public project / support page shown in the setup UI. */
10     export const SUPPORT_URL =
11         "https://github.com/avidullu/gmail-attachments-to-gdrive";
12
13     /** Privacy policy shown in the setup UI (and the OAuth consent screen). */
14     export const PRIVACY_POLICY_URL =
15         "https://github.com/avidullu/gmail-attachments-to-gdrive/blob/master/PRIVACY.md";
16
17     /**

```

```

18     * Apps Script enforces a daily quota on file creation. We cap ourselves below
19     * it so a single run can never exhaust the account's allowance.
20     */
21     export const MAX_FILES_ADD_PER_DAY = 240;
22
23     // Time constants, in milliseconds.
24     export const ONE_MINUTE = 60_000;
25     export const ONE_DAY = 86_400_000;
26     export const THIRTY_DAYS = 2_592_000_000;
27     export const ONE_YEAR = 31_622_400_000;
28     export const SIX_HOURS = 21_600_000;
29
30     /**
31     * Wall-clock budget for a single sync execution. Apps Script kills consumer
32     * scripts at 6 minutes, so we stop at 5 to leave margin for the final
33     * bookkeeping and trigger scheduling.
34     */
35     export const MAX_RUNTIME_MS = 5 * ONE_MINUTE;
36
37     /** Defensive upper bound on how many sync triggers may exist at once (limit is 20). */
38     export const MAX_SYNC_TRIGGERS = 12;
39
40     // Keys used in PropertiesService.getUserProperties().
41     export const PROP_FOLDER = "f";
42     export const PROP_LAST_SYNCED = "l";
43     export const PROP_FILES_ADDED_TODAY = "fa";
44     export const PROP_DEBUG = "dbg";
45     export const PROP_LAST_ERROR = "err";
46     export const PROP_PROCESSED_IDS = "pid";
47     export const PROP_RUN_STATS = "stats";
48     export const PROP_SUMMARY_LAST_SENT = "sumsent";
49     export const PROP_HISTORY_ID = "hid";
50
51     /** Flip to true to emit verbose Logger output during development. */
52     export const ENABLE_DEBUGGING = false;
53
54     /**
55     * Hybrid incremental sync. OFF by default so the proven date-window backfill
56     * remains the sole code path; flip to true to roll out the cheaper Gmail
57     * `history.list` steady state. When on, once a backfill catches up the current
58     * mailbox `historyId` is captured and subsequent runs process only newly-added
59     * messages, falling back to a date-window re-scan if the id expires (HTTP 404).
60     * Requires the Gmail advanced service (declared in appsscript.json); no extra
61     * OAuth scope ? `gmail.readonly` already covers history reads.
62     */
63     export const ENABLE_INCREMENTAL_SYNC = false;
64
65     /**
66     * Daily summary email. OFF by default ? enabling it requires the
67     * `script.send_mail` OAuth scope (a re-consent on next deploy). When enabled,
68     * set SUMMARY_EMAIL_RECIPIENT to the destination address (empty = the
69     * effective user's own address).
70     */
71     export const ENABLE_SUMMARY_EMAIL = false;
72     export const SUMMARY_EMAIL_RECIPIENT = "";
73

```

7. user (2026-06-10T16:27:30.999Z)

```

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\drive.test.ts:98:
expect(result.hasError).toBe(false);
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\query.test.ts:18:    const lastSynced
= new Date(2012, 0, 1).toJSON();
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\query.test.ts:19:    const r =
buildSearchQuery(lastSynced, now);

```

```

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\query.test.ts:22:    const after = new
Date(Date.parse(lastSynced) - ONE_DAY);
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\query.test.ts:23:    const before =
new Date(Date.parse(lastSynced) + THIRTY_DAYS + ONE_DAY);
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\query.test.ts:30:    const lastSynced
= new Date(2012, 0, 5).toJSON();
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\query.test.ts:31:    const r =
buildSearchQuery(lastSynced, now);
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\query.test.ts:40:    // lastSynced
ahead of now makes beforeQ === afterQ.
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\query.test.ts:41:    const lastSynced
= new Date(now.getTime() + 2 * ONE_DAY).toJSON();
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\query.test.ts:42:    const r =
buildSearchQuery(lastSynced, now);
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\syncIncremental.test.ts:128:
it("clears the cursor and rewinds lastSynced when the history expired", () => {
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\syncIncremental.test.ts:145:    //
lastSynced rewind to roughly 30 days ago for the recovery re-scan.
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\syncIncremental.test.ts:146:    const
lastSynced = Date.parse(props.store[PROP_LAST_SYNCED]);
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\syncIncremental.test.ts:147:
expect(Date.now() - lastSynced).toBeGreaterThan(29 * ONE_DAY);
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\syncIncremental.test.ts:153:    // No
stored historyId ? backfill mode; lastSynced=now means we are already
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\ui.test.ts:54:    // lastSynced is
roughly two years back.
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\ui.test.ts:55:    const lastSynced =
Date.parse(props.store[PROP_LAST_SYNCED]);
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\ui.test.ts:57:
expect(Math.abs(lastSynced - expected)).toBeLessThan(60_000);

```

8. user (2026-06-10T16:27:32.068Z)

```

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\state.ts:33:export function
getLastSynced(): string {
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\state.ts:37:export function
setLastSynced(iso: string): void {
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\sync.ts:99:    state.setLastSynced(new
Date(today.getTime() - 365 * ONE_DAY).toJSON());
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\sync.ts:133:    let lastSynced =
state.getLastSynced();
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\sync.ts:242:
state.setLastSynced(lastSynced);
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\ui.ts:66:    state.setLastSynced(new
Date(startMillis).toJSON());

```

9. user (2026-06-10T16:28:17.273Z)

Structured output provided successfully

10. assistant (2026-06-10T16:28:23.426Z)

Verification complete. The finding is real and confirmed at every link in the chain: the window advance ignores attachment-level errors, the failed messages are deliberately left for a retry that never comes (backfill only walks forward with a ~1-day overlap), and no other mechanism rewinds `lastSynced` in the default configuration. High severity stands, with the caveats that steady-state operation does retry for 1-2 days and the worst-case whole-window loss depends on the related cap-undercount finding.